

get the icon, resize, and place it such that it fits inside the actual size of the phone's display (5.4cm size in prototype). For the main screen UI, battery icon is used for getting power information, tour icon for getting the signal info, and dial icon for dialing screen, contacts, and messaging

screens. In the middle, it is kept blank to display time, date, etc.

Create the dialing screen UI for the numbers and other icons. Dial a number and go to home screen, or erase and refresh the screen. Similarly, make UIs for the remaining phone functions and screen.

Save the images from the UIs in the pic folder of the Python library that was cloned earlier. Open the code for displaying.

Coding phone OS

To create the code for OS for the phone, import the modules and library for interfacing the E-ink touch display. Then import the Python modules for using SIM 800L GSM module. Next, set the path where you keep the UI images for the phone and create the while loop to check the screen. The page to display is similar to previous, but new if its previous one has new page number. Update the display with UI pic.

For the (first) home page, check the touch points for the icons like dial icon, contact icon, or message icon. Also check the incoming call by checking the ring pin. Then create an 'if' condition where you check the touch points to match the range of icons on the UI. If you touch the phone icon, the touch point should match with the touch point corresponding to the phone.

Set the function in if condition to change the page number to next (No. 2). That is, in the UI image of dialing screen, update the phone screen and set the page number to 2. Now the phone moves to dialing screen with buttons of numbers. Create the if conditions for checking the numbers and icons.

Touch the dial pad screen if touch point range of number is one or two, or other number button. Update an empty string named number, add that number to the string. If touch point matches with the call icon, it sends the com-

```
elif(GT_Dev.X[0] > 40 and GT_Dev.X[0] < 60 and GT_Dev.Y[0] > 190 and GT_Dev.Y[0] < 220):
    print("call")
    gsm.Call(number)          # This call hangs up automatically after 15 seconds
    Page = 4

if(GT_Dev.X[0] > 30 and GT_Dev.X[0] < 60 and GT_Dev.Y[0] > 225 and GT_Dev.Y[0] < 250):
    print("home")
    image = Image.open(os.path.join(picdir, 'phone screen1.png'))
    epd.display(epd.getbuffer(image))
    Page=0

if(Page == 2):
    print("cintact")
    image = Image.open(os.path.join(picdir, 'contactsscreen.png'))
    epd.display(epd.getbuffer(image))

if(GT_Dev.X[0] > 30 and GT_Dev.X[0] < 60 and GT_Dev.Y[0] > 225 and GT_Dev.Y[0] < 250):
    print("home")
    Page=0
if(Page == 3):
    print("Message")

    image = Image.open(os.path.join(picdir, 'contactsscreen.png'))
    epd.display(epd.getbuffer(image))
    print("message c=screen")
    newSMS = gsm.SMS_read()
```

Fig. 12: Message function

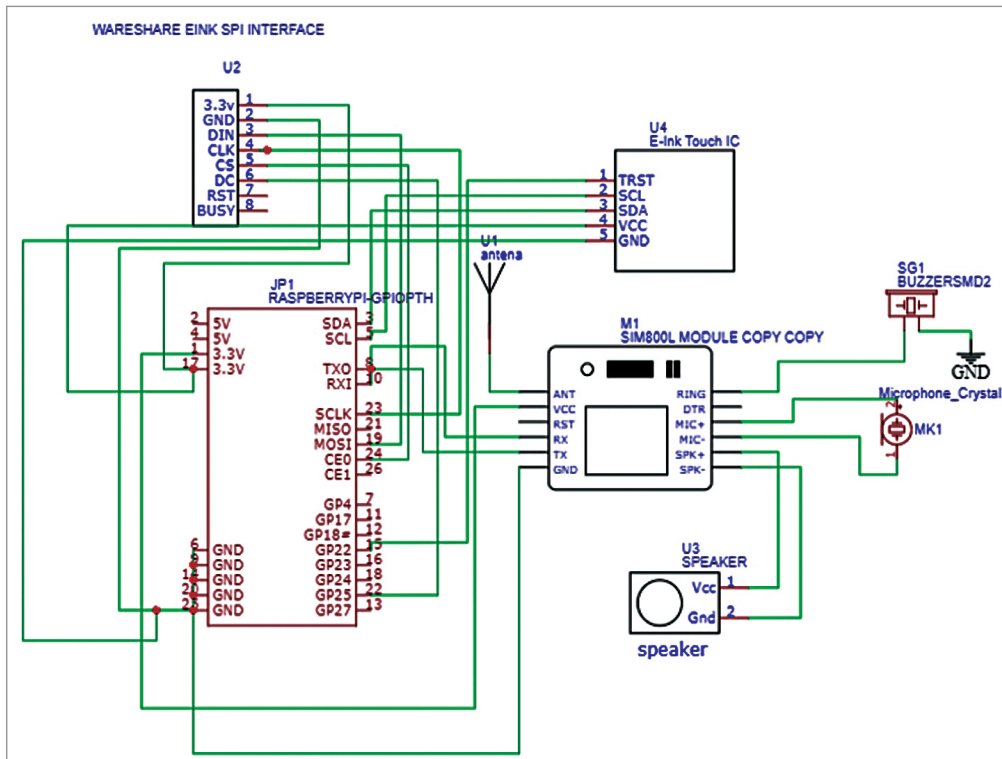


Fig. 13: Circuit diagram